

```

1  """
2  Example 4.7 Multiclass confusion matrix for the MNIST data set
3  Machine Learning for Engineering Problem Solving
4  @author: Austin Downey
5  """
6
7  import IPython as IP
8  IP.get_ipython().run_line_magic('reset', '-sf')
9
10 import numpy as np
11 import matplotlib.pyplot as plt
12 import sklearn as sk
13
14 cc = plt.rcParams['axes.prop_cycle'].by_key()['color']
15 plt.close('all')
16
17 %% Load your data
18
19 # Fetch the MNIST dataset from openml
20 mnist = sk.datasets.fetch_openml('mnist_784', as_frame=False, parser='auto')
21 X = np.asarray(mnist['data']) # load the data and convert to np array
22 Y = np.asarray(mnist['target'], dtype=int) # load the target
23
24 # Split the data set up into a training and testing data set
25 X_train = X[0:60000,:]
26 X_test = X[60000:,:]
27 Y_train = Y[0:60000]
28 Y_test = Y[60000:]
29
30 %% Confusion Matrix for a Multiclass classifier.
31
32 # Use the one-vs-one classifier that uses Stochastic Gradient Descent as this is
33 # faster for this specific data set
34 ovo_clf = sk.multiclass.OneVsOneClassifier(sk.linear_model.SGDClassifier())
35
36 # make a prediction for every case using the k-fold method.
37 Y_train_pred = sk.model_selection.cross_val_predict(ovo_clf, X_train, Y_train, cv=3)
38 conf_mx = sk.metrics.confusion_matrix(Y_train, Y_train_pred)
39 print(conf_mx)
40
41 # plot the results
42 fig = plt.figure(figsize=(4,4))
43 pos = plt.imshow(conf_mx) #, cmap=plt.cm.gray)
44 cbar = plt.colorbar(pos)
45 cbar.set_label('number of classified digits')
46 plt.ylabel('actual digit')
47 plt.xlabel('estimated digit')
48 plt.savefig('confusion_matrix', dpi=300)
49
50 # Normalize the confusion matrix by class size to compare error rates, not raw counts.
51 row_sums = conf_mx.sum(axis=1, keepdims=True)
52 norm_conf_mx = conf_mx / row_sums
53
54 # Next, we remove the high values along the diagonal. This is done by converting the
55 # confusion matrix to a float data type, and replacing everything on the diagonal with
56 # NaNs.
57 conf_mx_noise = np.asarray(norm_conf_mx, dtype=np.float32)
58 np.fill_diagonal(conf_mx_noise, np.NaN)
59
60 # plot the results but only consider the noise
61 fig = plt.figure(figsize=(4,4))
62 pos = plt.imshow(conf_mx_noise) #, cmap=plt.cm.gray)
63 cbar = plt.colorbar(pos)
64 cbar.set_label('normalized classification error')
65 plt.ylabel('actual digit')
66 plt.xlabel('estimated digit')
67 plt.savefig('confusion_matrix_error', dpi=300)

```